

Challenge 1

Problema

Considere que tem um conjunto de dados em vírgula flutuante precisão simples organizado como um cubo tri-dimensional:

```
float Cube[N][N][N];          // cubo tri-dimensional
```

Para cada fatia bidimensional do cubo ($Cube[x][y=0..N-1][z=0..N-1]$) pretende-se calcular a expressão abaixo, sendo os resultados guardados no vector

```
float Res[N];                  // vector Resultados
```

$$Res[x] = \sum_{y=0}^{N-1} \sum_{z=0}^{N-1} \frac{Cube[x][y][z] + 10.0}{\sqrt[2]{Cube[x][y][0]}}$$

O código abaixo implementa esta operação.

```
void P06Challenge1 (float *Cube, float *Res, int n) {
    int x, y, z;

    for (x = 0; x < n; ++x) {
        Res[x] = 0.f;
    }
    for (y = 0; y < n; ++y) {
        for (z = 0; z < n; ++z) {
            for (x = 0; x < n; ++x) {

                /*
                 Res[x] += (Cube[x][y][z] + 10.) / sqrtf(Cube[x][y][0]);
                 */
                Res[x] += (Cube[x*n*n + y*n + z] + 10.f) / sqrtf(Cube[x*n*n + y*n]);
            }
        }
    }
}
```

Exercício 1 – Copie o ficheiro `/share/cpar/SAC/Challenge1`, construa o executável e preencha a primeira linha da tabela abaixo para $N=512$.

Para tal deve escrever

```
sbatch Challenge.sh 1
```

Versão	Observações	T (ms)	#I (G)	N = 512		
				CPI	L1_DCM (M)	LD+SR (M)
Challenge1()	Versão inicial não otimizada.					
Challenge2()	Rearranjo dos loops (x,y,z)					
Challenge3()	Variáveis locais					
Challenge4()	Rearranjo da expressão					
Challenge0()	Versão otimizada pela equipa docente.					

Exercício 2 – Execute agora a versão otimizada pela equipa docente.

Para tal deve escrever

```
sbatch Challenge.sh 0
```

Verifique que os resultados são semelhantes à última linha da tabela.

Exercício 3 – Será que o desempenho pode ser otimizado reordenando os *loop for* do código? Experimente com uma reordenação que lhe pareça dar os melhores resultados. Use a função `P06Challenge2()` em `Challenge.c`. Obtenha os resultado escrevendo:

```
sbatch Challenge.sh 2
```

Atualize a respetiva linha na tabela e reflita sobre os valores obtidos.

Exercício 4 – Será que o desempenho pode ser otimizado usando variáveis locais para aqueles acessos que exibem grande localidade temporal? Use a função `P06Challenge3()` em `Challenge.c`. Obtenha o resultado escrevendo:

```
sbatch Challenge.sh 3
```

Atualize a respetiva linha na tabela e reflita sobre os valores obtidos.

Exercício 5 – Será que o número de operações aritméticas realizadas, e, consequentemente, o número de instruções executadas, pode ser reduzido simplificando a expressão matemática que é calculada?

Note que

$$\sum_{y=0}^{N-1} \sum_{z=0}^{N-1} \frac{Cube[x][y][z] + 10.0}{\sqrt[2]{Cube[x][y][0]}} = \sum_{y=0}^{N-1} \left[\frac{1.0}{\sqrt[2]{Cube[x][y][0]}} * \left(10.0 * N + \sum_{z=0}^{N-1} Cube[x][y][z] \right) \right]$$

Use a função `P06Challenge4()` em `Challenge.c`. Obtenha o resultado escrevendo:

```
sbatch Challenge.sh 4
```

Atualize a respetiva linha na tabela e reflita sobre os valores obtidos.